SMART SHOP CREDIT TRACKER – JAVA DEVELOPMENT

A SUMMER INTERNSHIP REPORT

Submitted by

PARAMESHWARAN C – 810023205305



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report **"Smart Shop Credit Tracker – Java Development"** is the Bonafide work of **PARAMESHWARAN C (810023205305)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026**. The internship was carried out at Astonish InfoTech Private Limited, Tiruchirappalli from 01-06-2026 to 30-06-2026.

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The **Smart Shop Credit Tracker** is a web-based customer credit and payment management application developed using **Java 17, Spring Boot and a MySQL database**. It was developed during the Summer Internship at **Astonish InfoTech Private Limited, Tiruchirappalli** from 01/06/2026 to 30/06/2026 in the domain of Java Development. The system gives a small shop owner a simple and reliable replacement for the traditional credit notebook by managing **customer details, credit purchases, payments received, customer ledgers, pending dues and date-wise reports**.

The internship focused on learning **Object-Oriented Programming (OOP), Exception Handling, the Collections Framework, JDBC database connectivity and the Spring Boot framework**. These concepts were applied in the project through record-based model classes (Customer and LedgerEntry), validated request classes, a ShopController that exposes the REST APIs, a ShopService that holds the business rules, a ShopRepository that accesses MySQL through Spring JdbcTemplate, and Spring Security for staff login. A browser-based interface (dashboard, customers, credit entries, payments, customer ledger, pending dues and reports) uses these APIs.

The application stores its data in a MySQL database named **smart_shop_db**, which contains four tables – **customers**, **credit_entries**, **payments** and **users**. The outstanding balance of a customer is always calculated from the saved transactions (total credit minus total payments), so it never goes out of sync. The system rejects payments larger than the outstanding balance, blocks future-dated transactions, prevents new credit for archived customers, protects against duplicate payment submission using a unique request key, and secures access with BCrypt-hashed passwords, session login and CSRF protection.

Overall, the project successfully implements **Create, Read, Update and Delete (CRUD)** operations on customer records together with credit, payment, ledger and report modules, and demonstrates the integration of a Spring Boot REST application with a relational database. It highlights how Java can be used to build secure, accurate and practical business applications for small retail shops.

Keywords: Java, Spring Boot, REST API, JDBC, MySQL, Spring Security, Credit Management, Customer Ledger, CRUD Operations

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVES OF THE INTERNSHIP 1.2 MOTIVATION	1
2	COMPANY PROFILE	3
3	DAILY WORK REPORT	4
4	BLOCK DIAGRAM	7
5	SMART SHOP CREDIT TRACKER 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILITIES 5.4 SYSTEM REQUIREMENTS 5.5 ARCHITECTURAL DESIGN 5.6 DATABASE DESIGN	8
6	PROPOSED METHOD 6.1 AUTHENTICATION AND ACCESS CONTROL 6.2 CUSTOMER MANAGEMENT 6.3 CREDIT ENTRY MANAGEMENT 6.4 PAYMENT MANAGEMENT 6.5 CUSTOMER LEDGER 6.6 PENDING DUES AND REPORTS 6.7 DATABASE CONNECTIVITY 6.8 VALIDATION AND EXCEPTION HANDLING	13
7	ALGORITHM 7.1 USER LOGIN 7.2 ADD CUSTOMER 7.3 RECORD CREDIT ENTRY 7.4 RECORD PAYMENT 7.5 VIEW CUSTOMER LEDGER 7.6 VIEW PENDING DUES 7.7 GENERATE REPORT	15
8	CODE IMPLEMENTATION	17
9	SAMPLE OUTPUT	24
10	ANALYSIS	28
11	CONCLUSION	30
12	FUTURE WORK	30
13	REFERENCES	30

1. INTRODUCTION

The **Smart Shop Credit Tracker** is a Java web application developed to automate and simplify the management of customer credit in small retail shops. Many local shops sell goods on credit to regular customers and note the amounts in a handwritten notebook. Such records are easily lost, difficult to search, hard to total and prone to calculation mistakes and disputes between the shopkeeper and the customer. To overcome these challenges, a computerized **Smart Shop Credit Tracker** has been developed.

The primary purpose of this project is to provide a reliable and easy-to-use application where the shop owner can add customers, record credit purchases, record payments received through cash or UPI, view the complete ledger of each customer with a running balance, see the list of customers with pending dues, and generate reports for any date range. The system keeps every balance accurate, validates every entry and stores all records permanently in a database, which reduces manual work and avoids calculation errors.

The back end of the application is developed using **Java 17 and Spring Boot 3.5.6**. The REST APIs are built with Spring Web, and the data is accessed using **JDBC through Spring JdbcTemplate** with the **MySQL Connector/J** driver and a **MySQL 8.0** relational database. Request validation is done with Jakarta Bean Validation, and staff login is secured with **Spring Security** using BCrypt password hashing, session management and CSRF protection. A browser-based web interface communicates with the back end through JSON APIs. **JDK 17**, **VS Code** and **Maven** were used for development, and the **MySQL command-line client** was used for database administration.

The system follows a **three-tier architecture**, consisting of the Presentation Layer (the web interface and the ShopController), the Business Logic Layer (the ShopService class) and the Data Access Layer (the ShopRepository class with the MySQL database accessed through JDBC). This layered structure keeps the code modular, easy to maintain and easy to extend with new features in future versions.

The project is divided into several modules, including **Authentication, Customer Management, Credit Entries, Payments, Customer Ledger, Pending Dues and Reports**. Each module performs a specific function and together they provide a complete credit management solution for a shop. Overall, the project is a practical implementation of Core Java, Spring Boot, JDBC and MySQL that provides hands-on knowledge of OOP, exception handling, REST API development, database connectivity and software development in a

professional environment. It can be further enhanced with features such as SMS reminders, PDF statements, multiple staff roles and cloud deployment.

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the Smart Shop Credit Tracker is to develop a complete, reliable and efficient credit management application using Java, Spring Boot and MySQL. The system is designed to simplify the handling of customer credit by providing an automated platform to manage customers, credit entries, payments and reports effectively. The objectives of the internship are:

- To gain practical knowledge and hands-on experience in the **Core Java** programming language.
- To understand and implement the four pillars of **Object-Oriented Programming** – Encapsulation, Inheritance, Polymorphism and Abstraction.
- To learn and apply **Exception Handling** techniques, including custom exceptions and a central error handler, for robust error management.
- To understand the **Java Collections Framework** including ArrayList, HashMap and LinkedList.
- To learn database connectivity using **JDBC with the MySQL database**.
- To learn **Spring Boot** and build a layered **REST API** using controller, service and repository classes.
- To understand the basics of application security – **authentication, password hashing and CSRF protection**.
- To understand **transaction management and data integrity** in financial applications.
- To develop a complete real-world **shop credit management application** using Java, Spring Boot and MySQL.
- To improve **problem-solving skills** through practical programming exercises.
- To gain experience of working in a **professional IT environment**.

1.2 MOTIVATION

The motivation behind developing the **Smart Shop Credit Tracker** is to make credit handling in small shops faster, more accurate and more reliable. Many shops still depend on handwritten notebooks to remember who has taken goods on credit and how much has been paid, which

can lead to forgotten entries, calculation mistakes, lost pages and arguments over the amount due.

This project provides a computerized system in which customers can be added, credit purchases and payments can be recorded with their date, and the outstanding balance of every customer is calculated instantly from the saved transactions. It also validates operations such as payments, so that a customer can never be shown as paying more than the amount actually due.

Another major motivation is that Java and Spring Boot are widely used in industry for building secure and maintainable business applications. Building a practical application with **Java, Spring Boot, JDBC and MySQL** therefore provides valuable knowledge of OOP, REST API design, database connectivity and security, and demonstrates how modern software technologies can be applied to solve a real-world problem faced by local shopkeepers.

2. COMPANY PROFILE

About the Company:

Astonish InfoTech Private Limited is an Information Technology (IT) company located in Tiruchirappalli. The company was established with the vision of providing innovative and quality IT solutions to clients across various domains. It specializes in a comprehensive range of IT services including Web Design, Web Development, Web Hosting, Domain Registration, Digital Marketing, Android Application Development and Java Development, and has a dedicated team of experienced professionals committed to delivering high-quality solutions.

Astonish InfoTech has a strong commitment to student development and provides internship and training opportunities to engineering students. These programs help students bridge the gap between academic learning and industry requirements by giving them hands-on exposure to real-world software development projects.

As part of my internship at Astonish InfoTech Private Limited, I worked in the domain of **Java Development** and developed a **Smart Shop Credit Tracker** using Java, Spring Boot and a MySQL database. The internship covered Core Java fundamentals, Object-Oriented Programming, Exception Handling, the Collections Framework, JDBC and MySQL and the Spring Boot framework, followed by the design, implementation and testing of the credit tracking application. The system was designed to simplify credit management in a shop through features such as customer management, credit entries, payments, customer ledger, pending dues and reports.

The internship gave me valuable practical knowledge of Java development and helped me understand the complete Software Development Life Cycle (SDLC). It improved my programming skills, database handling, problem-solving ability and understanding of how a Java web application is integrated with a relational database. Overall, the internship provided an excellent opportunity to apply theoretical concepts in a practical software development environment.

Company Details:

- **Company Name:** Astonish InfoTech Private Limited
- **Industry:** Information Technology (IT) Services and Software Development
- **Address:** G7, Technology Park, Bharathidasan University (BUTP), Kajamalai Campus, Tiruchirappalli – 620 023, Tamil Nadu, India
- **Phone:** 0431-2331318, 09994954031
- **Email:** admin@astonishinfotech.com
- **Website:** www.astonishinfotech.com
- **Internship Domain:** Java Development
- **Project Completed During Internship:** Smart Shop Credit Tracker (Java Development)

Services Offered

- Web Design
- Web Development
- Web Hosting
- Domain Registration
- Digital Marketing
- Android Application Development
- Java Development
- Internship and Student Training Programs

Technologies Used During the Internship

- **Programming Language:** Java (JDK 17)
- **Framework:** Spring Boot 3.5.6 (Spring Web, Spring Security, Validation)
- **Database:** MySQL Server 8.0
- **Database Connectivity:** JDBC with Spring JdbcTemplate and MySQL Connector/J

- **Build Tool:** Apache Maven (Maven Wrapper)
- **Development Tool:** VS Code
- **Database Administration Tool:** MySQL command-line client
- **Operating System:** Windows 11

3. DAILY WORK REPORT

Day 1: Attended the orientation session at Astonish InfoTech; briefed on internship guidelines, evaluation criteria and reporting format. Discussed and finalized the project topic – "Smart Shop Credit Tracker" – with the mentor.

Day 2: Set up the development environment – installed JDK 17, VS Code, Maven, MySQL Server 8.0 and the MySQL command-line client. Verified the installation with a sample "Hello World" program.

Day 3: Studied Core Java basics – structure of a Java program, variables, data types and operators – and practiced simple programs.

Day 4: Learned control statements (if-else, switch) and looping statements (for, while, do-while) with practice programs.

Day 5: Studied arrays and Strings and solved practice problems on array operations and string handling.

Day 6: Learned methods in Java – method definition, parameters, return types and method overloading.

Day 7: Revised the Week 1 topics, completed practice programs and reviewed progress with the mentor.

Day 8: Started Object-Oriented Programming – classes, objects and constructors.

Day 9: Studied Encapsulation – private data members with getters and setters – and practiced with sample classes.

Day 10: Learned Inheritance – types of inheritance and the use of the super keyword.

Day 11: Studied Polymorphism – method overloading and method overriding.

Day 12: Learned Abstraction – abstract classes and interfaces.

Day 13: Solved practice programs combining all four pillars of OOP.

Day 14: Revised OOP concepts and completed mini programs; weekly review with the mentor.

Day 15: Studied Exception Handling – try, catch, finally, throw and throws – and handled common runtime errors.

Day 16: Learned the Collections Framework – ArrayList and LinkedList – with practice programs.

Day 17: Studied HashMap and iterating over collections; practiced storing and retrieving data.

Day 18: Learned MySQL basics – creating databases and tables, joins, SUM and GROUP BY – and executed CRUD queries.

Day 19: Studied JDBC – architecture, drivers and the steps to connect Java with MySQL – and practiced Connection, PreparedStatement and ResultSet programs.

Day 20: Learned the basics of Spring Boot – project structure, Maven dependencies, application.properties and a simple REST controller – and ran the first API.

Day 21: Revised the Week 3 topics; carried out requirement analysis for the Smart Shop Credit Tracker and listed the required modules – customers, credit entries, payments, ledger, dues and reports.

Day 22: Designed the database 'smart_shop_db' and created the customers, credit_entries, payments and users tables with primary keys, foreign keys and constraints in MySQL.

Day 23: Designed the class structure and implemented the Customer and LedgerEntry model records and the request classes with validation rules (CustomerRequest, CreditRequest, PaymentRequest).

Day 24: Implemented the ShopRepository using JdbcTemplate and configured the MySQL datasource; developed the customer insert, update, search and status methods.

Day 25: Developed the repository methods for credit entries and payments, the balance calculation query and the customer ledger with running balance.

Day 26: Implemented the ShopService business rules – payment limit check, future-date check, archived-customer check, customer row locking and duplicate payment protection.

Day 27: Developed the ShopController REST endpoints, the dashboard, pending dues and report queries, and the central ApiExceptionHandler for error responses.

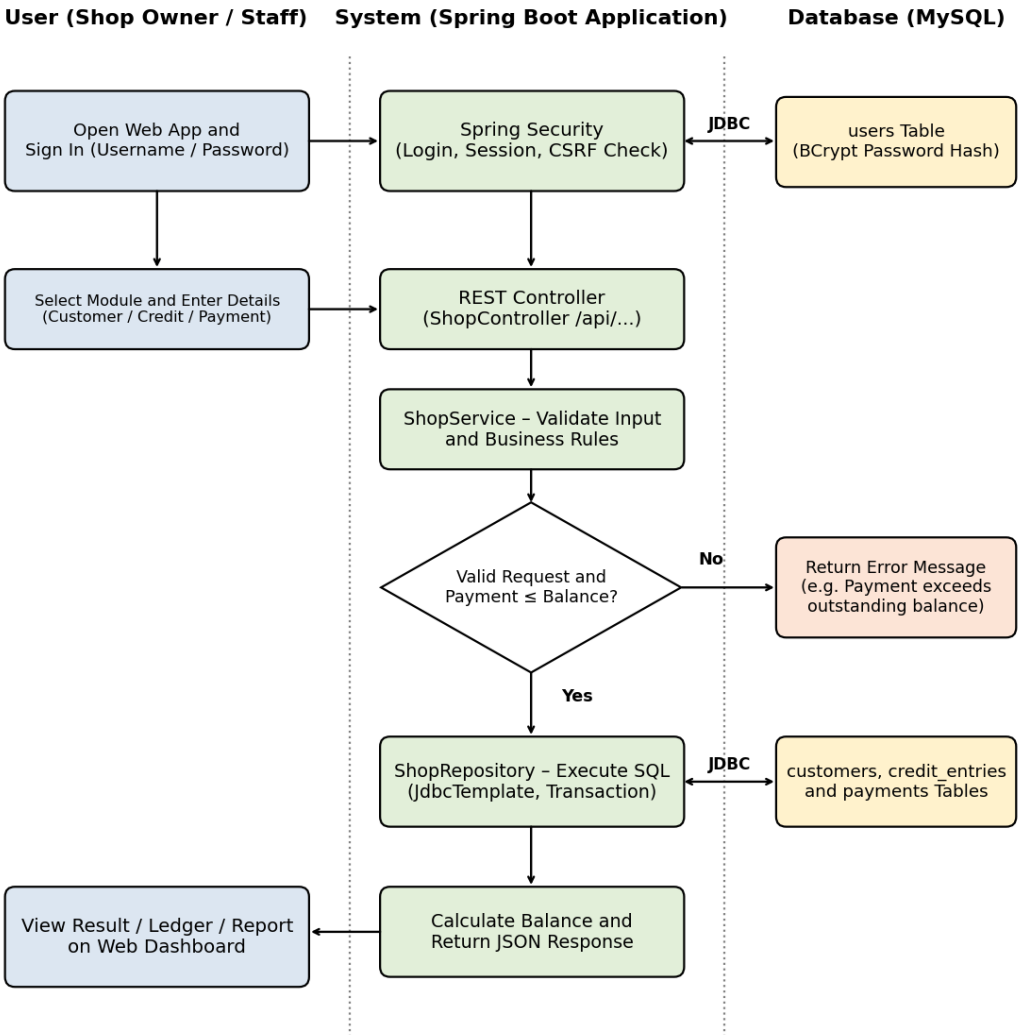
Day 28: Implemented Spring Security – BCrypt password hashing, session login, CSRF protection and the bootstrap admin user – and connected the web interface to the APIs.

Day 29: Tested all modules through the web interface and with JUnit and Mockito test cases, fixed the identified bugs, captured output screenshots and verified the data in the MySQL tables.

Day 30: Compiled the final internship report and presented the project to the mentor.

4. BLOCK DIAGRAM

The block diagram below shows the overall working of the Smart Shop Credit Tracker. The shop owner signs in through the web interface, where Spring Security verifies the credentials against the users table. After login, the user selects a module (customer, credit, payment, ledger, dues or report) and enters the required details. The request reaches the ShopController as a REST call and is passed to the ShopService, which validates the input and applies the business rules, such as rejecting a payment that is larger than the outstanding balance. Valid requests are executed on the customers, credit_entries and payments tables by the ShopRepository through JDBC, the balance is recalculated, and the result is returned as a JSON response and displayed in the web interface. Invalid requests return a clear error message.



5. SMART SHOP CREDIT TRACKER

5.1 Existing System

Most small shops still manage customer credit through manual or semi-manual methods. An analysis of the existing approaches reveals the following:

- **Handwritten Credit Notebooks** – the shopkeeper writes the customer name, items and amount in a notebook. These records are easily lost or damaged, entries can be missed or written wrongly, totals must be calculated by hand, and searching for one customer's history takes a long time.
- **Spreadsheet-Based Records** – some shops use Excel or similar sheets, which are better than paper. However, balances still depend on manual formulas, there is no validation, any user can edit or delete past entries by mistake, and there is no login or access control.
- **Full Billing and ERP Software** – commercial billing packages offer many features, but they are often costly and complex for a small shop, include stock and tax modules that are not needed, and need training before they can be used.

Based on this analysis, the following problems were identified in the existing credit management practice:

- **Manual calculation of balances** is time-consuming and prone to human errors.
- **Difficulty in finding the full credit and payment history** of a particular customer.
- **Risk of data loss** because notebooks and loose sheets can be lost, damaged or altered.
- **No instant view of pending dues**, so it is hard to know who owes money and for how long.
- **Lack of validation** – a payment larger than the amount due can be written without any warning.
- **Duplicate or wrong entries** cannot be detected easily.
- **Difficulty in generating reports** for a period such as a week or a month.
- **No security or access control**, so anyone with access to the notebook or file can view or change the data.

These drawbacks make the existing manual system less efficient as the number of customers and transactions grows. Therefore, there is a need for a computerized Smart Shop Credit Tracker that provides a reliable, simple and secure platform for managing customer credit.

5.2 Proposed System

The proposed **Smart Shop Credit Tracker** is developed using **Java, Spring Boot and MySQL**. It replaces the credit notebook with an integrated web application that stores all customer, credit and payment data in a centralized database. Object-Oriented Programming principles and a layered architecture are applied throughout the development to ensure code modularity and maintainability. The development followed a structured approach including requirement analysis, database design, class design, implementation and testing. Key aspects of the proposed system include:

- **Centralized customer records** – name, phone number, address and active status of every customer are stored in one place.
- **Credit entries** – every credit purchase is saved with its description, amount, date and optional notes.
- **Payment recording** – payments are saved with the method (Cash, UPI or Other) and are limited to the outstanding balance.
- **Automatic balance calculation** – the balance is calculated from the saved transactions, so it is always correct.
- **Customer ledger** – a date-wise statement of credits and payments with a running balance.
- **Pending dues** – a list of customers who still owe money, with the oldest credit date and the number of days.
- **Reports** – credit transactions, payment collections and outstanding balances, with date-range filters and CSV export.
- **Secure login** – staff access with BCrypt-hashed passwords, session management and CSRF protection.
- **Validation and duplicate protection** – invalid inputs, future dates and repeated payment submissions are rejected.

By automating these processes, the proposed system reduces manual work, improves data accuracy and provides a convenient and efficient way of managing customer credit in a shop.

5.3 Capabilities

The Smart Shop Credit Tracker offers the following key capabilities:

- **Customer Management** – Adds new customers, updates their details, searches by name or phone number, and archives or reactivates customers without deleting their history.

- **Credit Entries** – Records items sold on credit with amount, date and notes, and lists recent credit entries.
- **Payments** – Records cash, UPI or other payments and rejects any payment that exceeds the outstanding balance.
- **Customer Ledger** – Shows all credits and payments of a customer in date order with the running balance and totals.
- **Dashboard** – Displays active customers, total credit, total payments, outstanding amount, customers with dues and the ten most recent transactions.
- **Pending Dues** – Lists customers with an outstanding balance, the oldest credit date and the days since that date.
- **Reports** – Generates credit, payment and outstanding balance reports for a selected date range and exports them to CSV.
- **Security and Error Handling** – Provides login protection and clear messages for invalid or unsuccessful operations.

5.4 System Requirements

5.4.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher
RAM	Minimum 4 GB
Hard Disk	Minimum 20 GB free space
Operating System	Windows 10/11 or Linux
Monitor	Standard Display

5.4.2 Software Requirements

Software	Version / Details
JDK	Java Development Kit 17
Framework	Spring Boot 3.5.6
Build Tool	Apache Maven (Maven Wrapper)
Database	MySQL Server 8.0
JDBC Driver	MySQL Connector/J
Data Access	Spring JdbcTemplate
Security	Spring Security (BCrypt)
IDE	VS Code
DB Client	MySQL command-line client

Software	Version / Details
Browser	Google Chrome or Microsoft Edge
OS	Windows 11

5.5 Architectural Design

The Smart Shop Credit Tracker follows a three-tier architecture:

- **Presentation Layer (Tier 1)** – The browser-based web interface and the ShopController and AuthController classes, which receive the requests and return JSON responses.
- **Business Logic Layer (Tier 2)** – The ShopService class, containing all the business rules such as payment limit checking, date validation, customer locking and duplicate payment protection.
- **Data Access Layer (Tier 3)** – The ShopRepository class using Spring JdbcTemplate and the MySQL database for persistent data storage and retrieval.

The data flow in the system is as follows:

- The user signs in and selects an operation in the web interface.
- The interface sends a REST request to the ShopController, which checks the input using validation rules.
- The ShopController calls the matching ShopService method.
- The ShopService applies the business rules and calls the ShopRepository.
- The ShopRepository executes parameterized SQL queries on the customers, credit_entries and payments tables through JDBC.
- The results are returned to the controller as Java objects and sent to the interface as a JSON response.
- Errors are converted into a standard error response by the ApiExceptionHandler.

5.6 Database Design

The MySQL database named ‘smart_shop_db’ contains four tables designed to store all the shop credit information. The balance of a customer is not stored in any column; it is calculated as the total of the credit entries minus the total of the payments.

5.6.1 Customers Table

Column	Data Type	Constraint	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique Customer ID

Column	Data Type	Constraint	Description
full_name	VARCHAR(120)	NOT NULL	Customer Name
phone	VARCHAR(20)	NOT NULL	Phone Number
address	VARCHAR(255)	NULL	Customer Address
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Active / Archived Status
created_at	DATETIME	NOT NULL	Record Creation Time
updated_at	DATETIME	NOT NULL	Last Update Time

5.6.2 Credit Entries Table

Column	Data Type	Constraint	Description
id	BIGINT	PK, AUTO_INCREMENT	Credit Entry ID
customer_id	BIGINT	FK, NOT NULL	Customer Reference
description	VARCHAR(255)	NOT NULL	Items / Details
amount	DECIMAL(12,2)	NOT NULL	Credit Amount
transaction_date	DATE	NOT NULL	Date of Credit
notes	VARCHAR(500)	NULL	Additional Notes
created_at	DATETIME	NOT NULL	Record Creation Time

5.6.3 Payments Table

Column	Data Type	Constraint	Description
id	BIGINT	PK, AUTO_INCREMENT	Payment ID
customer_id	BIGINT	FK, NOT NULL	Customer Reference
amount	DECIMAL(12,2)	NOT NULL	Payment Amount
transaction_date	DATE	NOT NULL	Date of Payment
payment_method	VARCHAR(20)	NOT NULL	CASH / UPI / OTHER
notes	VARCHAR(500)	NULL	Additional Notes
request_key	VARCHAR(80)	UNIQUE, NOT NULL	Duplicate Submission Key
created_at	DATETIME	NOT NULL	Record Creation Time

5.6.4 Users Table

Column	Data Type	Constraint	Description
username	VARCHAR(50)	PK	Login User Name
password_hash	VARCHAR(100)	NOT NULL	BCrypt Password Hash
role	VARCHAR(20)	NOT NULL	User Role (ADMIN)

Column	Data Type	Constraint	Description
active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account Enabled Status

6. PROPOSED METHOD

6.1 Authentication and Access Control

- The user signs in with a username and password through the /api/auth/login endpoint.
- Passwords are stored in the users table only as BCrypt hashes, never as plain text.
- After a successful login a session is created and the session ID is changed to prevent session fixation.
- A CSRF token is issued as a cookie and must be sent in the X-XSRF-TOKEN header with every modifying request.
- All other /api endpoints require authentication; unauthenticated requests receive a 401 response.
- The first admin user is created at start-up only when the users table is empty and a password of at least 12 characters is supplied through an environment variable.
- Cross-origin access (CORS) is limited to the local web interface address.

6.2 Customer Management

- Adds a new customer by capturing the full name, phone number and address.
- Validates the phone number format and the maximum length of every field.
- Allows the customer details to be updated and customers to be searched by name or phone number.
- Archives or reactivates a customer instead of deleting the record, so that the history is preserved.
- Archived customers cannot receive new credit entries.

6.3 Credit Entry Management

- Records a credit purchase with the customer, description, amount, date and optional notes.
- The amount must be at least 0.01 with at most two decimal places; the date defaults to today and future dates are rejected.
- The customer row is locked (SELECT … FOR UPDATE) while the entry is saved so that simultaneous entries cannot disturb the balance.
- Returns the updated balance of the customer after the entry is saved.
- Lists the credit entries of a customer and the 100 most recent credit entries of the shop.

6.4 Payment Management

- Records a payment with the customer, amount, date, method (CASH, UPI or OTHER) and optional notes.
- Rejects a payment that is greater than the current outstanding balance of the customer.
- Every payment carries a unique request key; if the same key is submitted again for the same customer, the existing payment is returned instead of saving a duplicate.
- A request key that was already used for another customer is rejected.
- Returns the updated balance after the payment is saved.

6.5 Customer Ledger

- Combines the credit entries and payments of a customer into one list ordered by date.
- On the same date, credits are listed before payments.
- Calculates the running balance after every entry and returns the customer details with the total credit, total payments and current balance.

6.6 Pending Dues and Reports

- The dashboard shows the number of active customers, total credit, total payments, total outstanding amount, customers with dues and the ten most recent transactions.
- The pending dues report lists every customer with a balance greater than zero, along with the oldest credit date and the days since it. This date is an estimate because payments are not allocated to individual purchases.
- The outstanding balances report shows the total credit, total payments and balance of each customer who owes money.
- The credit and payment reports list the transactions between two selected dates; the start date must not be after the end date and future dates are not allowed.
- The report results can be exported to a CSV file.

6.7 Database Connectivity

- Spring Boot creates the MySQL datasource from application.properties; the database URL, user name and password are read from environment variables so that no password is stored in the code.
- All SQL operations are executed using Spring JdbcTemplate with parameterized queries (? placeholders), which also protects against SQL injection.
- Spring obtains and releases the database connections automatically, and service methods that change data are marked @Transactional.

6.8 Validation and Exception Handling

- Input is validated using Jakarta Bean Validation annotations such as @NotBlank, @Positive, @DecimalMin, @Digits, @Size and @Pattern.
- Business rule violations raise a BusinessException (HTTP 409) and missing records raise a NotFoundException (HTTP 404).
- A central ApiExceptionHandler converts all exceptions into a uniform JSON error response with the status, message and field errors.
- Internal error details are hidden from the user (server.error.include-message=never).

7. ALGORITHM

7.1 User Login – Algorithm

1. User opens the web application and enters the username and password.
2. The browser sends the credentials to the login API along with the CSRF token.
3. Spring Security loads the user from the users table and compares the password with the stored BCrypt hash.
4. If the credentials are wrong, the message “Username or password is incorrect” is returned.
5. If they are correct, a new session is created and the user details are returned.
6. The dashboard is displayed to the user.

7.2 Add Customer – Algorithm

1. User selects “Customers” and enters the name, phone number and address.
2. The request is validated for a non-empty name, a valid phone number and the maximum field lengths.
3. If the validation fails, the field errors are returned to the user.
4. ShopRepository executes the INSERT query on the customers table.
5. The saved customer record is read back with the totals and returned to the user.

7.3 Record Credit Entry – Algorithm

1. User selects “Credit entries” and enters the customer, description, amount, date and notes.
2. The input is validated and the date is checked so that it is not in the future.
3. The customer row is locked; if the customer is not found, an error is returned.
4. If the customer is archived, the entry is rejected.
5. ShopRepository executes the INSERT query on the credit_entries table.
6. The updated balance of the customer is calculated and “Credit recorded” is displayed.

7.4 Record Payment – Algorithm

1. User selects “Payments” and enters the customer, amount, date, payment method and notes; a unique request key is generated for the form.
2. The input and the date are validated and the customer row is locked.
3. If the request key already exists for the same customer, the existing payment is returned as “already recorded”.
4. The current balance is calculated as total credit minus total payments.

5. If the amount is greater than the balance, the payment is rejected with an error message.
6. Otherwise, the payment is inserted into the payments table and the new balance is returned.

7.5 View Customer Ledger – Algorithm

1. User selects “Customer ledger” and chooses a customer.
2. The system reads the customer details and totals.
3. The credit entries and payments of the customer are combined and sorted by date, with credits before payments on the same date.
4. The running balance is calculated for each entry by adding credits and subtracting payments.
5. The ledger with the entries and the final balance is displayed.

7.6 View Pending Dues – Algorithm

1. User selects “Pending dues”.
2. ShopRepository calculates the total credit, total payments and the oldest credit date of every customer.
3. Only customers whose balance is greater than zero are selected and sorted by the highest balance.
4. The number of days since the oldest credit date is calculated.
5. The pending dues list is displayed and can be filtered by name or phone number.

7.7 Generate Report – Algorithm

1. User selects “Reports”, chooses the report type and the From and To dates.
2. The system checks that the start date is not after the end date and that the dates are not in the future.
3. ShopRepository executes the matching query for credit transactions, payment collections or outstanding balances.
4. The result rows are displayed in a table.
5. If requested, the result is exported to a CSV file.

8. CODE IMPLEMENTATION

8.1 Customer Record (Model)

The Customer class is a Java record that represents the customer entity with its totals and balance.

```
public record Customer(
        long id,
        String fullName,
        String phone,
        String address,
        boolean active,
        LocalDateTime createdAt,
        BigDecimal totalCredit,
        BigDecimal totalPayments,
        BigDecimal balance) {
}
```

8.2 CreditRequest – Input Validation

The CreditRequest class receives the credit entry details and validates them using Bean Validation annotations.

```
public record CreditRequest(
        @Positive long customerId,
        @NotBlank @Size(max = 255) String description,
        @NotNull @DecimalMin(value = "0.01")
        @Digits(integer = 10, fraction = 2) BigDecimal amount,
        LocalDate transactionDate,
        @Size(max = 500) String notes) {
}
```

8.3 Database Connection, User and Password

The MySQL connection is set in application.properties; the database user name and password are read from environment variables, so no password is stored in the code.

```
spring.datasource.url=${SHOP_DB_URL:jdbc:mysql://localhost:3306/smart_shop_db
    ?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC}
spring.datasource.username=${SHOP_DB_USERNAME:smart_shop_app}
spring.datasource.password=${SHOP_DB_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
shop.bootstrap.username=${SHOP_BOOTSTRAP_USERNAME:admin}
shop.bootstrap.password=${SHOP_BOOTSTRAP_PASSWORD:}
CREATE USER 'smart_shop_app'@'localhost' IDENTIFIED BY '<database-password>';
GRANT SELECT, INSERT, UPDATE ON smart_shop_db.* TO
'smart_shop_app'@'localhost';
$env:SHOP_DB_PASSWORD = "<database-password>"
$env:SHOP_BOOTSTRAP_PASSWORD = "<admin-password-min-12-chars>"
```

8.4 ShopService – Add Credit

Validates the date, locks the customer, rejects archived customers and saves the credit entry.

```
@Transactional
public Map<String, Object> addCredit(CreditRequest request) {
    validateTransactionDate(request.transactionDate());
    if (!repository.lockCustomer(request.customerId())) {
        throw new NotFoundException("Customer not found");
    }
    Customer customer = customer(request.customerId());
    if (!customer.active()) {
        throw new BusinessException(
            "Archived customers cannot receive new credit entries");
    }
    long id = repository.insertCredit(request,
        request.transactionDate() == null
            ? LocalDate.now() : request.transactionDate());
    return Map.of("id", id,
        "customerId", request.customerId(),
        "balance", repository.getBalance(request.customerId()),
        "message", "Credit recorded");
}
```

8.5 ShopService – Add Payment

Checks for a duplicate submission, validates the payment against the outstanding balance and saves it (shortened for readability).

```
@Transactional
public Map<String, Object> addPayment(PaymentRequest request) {
    validateTransactionDate(request.transactionDate());
    if (!repository.lockCustomer(request.customerId())) {
        throw new NotFoundException("Customer not found");
    }
    Long priorCustomerId = repository
        .findPaymentCustomerIdByRequestKey(request.requestKey());
    if (priorCustomerId != null) {
        if (priorCustomerId != request.customerId()) {
            throw new BusinessException("This request key was "
                + "already used for another customer");
        }
        // duplicate submission: return the existing payment
        return Map.of("message", "This payment was already recorded");
    }
    BigDecimal currentBalance =
        repository.getBalance(request.customerId());
    if (request.amount().compareTo(currentBalance) > 0) {
        throw new BusinessException("Payment exceeds the current "
            + "outstanding balance of " + currentBalance);
    }
    long id = repository.insertPayment(request, paymentDate);
    return Map.of("id", id, "message", "Payment recorded",
        "balance", repository.getBalance(request.customerId()));
}
```

8.6 ShopRepository – Balance and Ledger

The balance is calculated from the saved transactions, and the ledger adds a running balance to every entry.

```
public BigDecimal getBalance(long customerId) {
    return jdbc.queryForObject("""
        SELECT COALESCE((SELECT SUM(amount) FROM credit_entries
                         WHERE customer_id = ?), 0.00)
             - COALESCE((SELECT SUM(amount) FROM payments
                         WHERE customer_id = ?), 0.00)
        """, BigDecimal.class, customerId, customerId);
}

public boolean lockCustomer(long id) {
    return !jdbc.query(
        "SELECT id FROM customers WHERE id = ? FOR UPDATE",
        (rs, row) -> rs.getLong(1), id).isEmpty();
}

// inside getLedger(): running balance
BigDecimal running = BigDecimal.ZERO;
for (LedgerEntry entry : entries) {
    running = running.add(entry.credit())
                     .subtract(entry.payment());
    result.add(new LedgerEntry(entry.id(), entry.transactionDate(),
        entry.type(), entry.description(), entry.credit(),
        entry.payment(), running));
}
```

8.7 ShopController – REST Endpoints

The ShopController exposes the REST APIs used by the web interface (selected endpoints).

```
@RestController
@RequestMapping("/api")
public class ShopController {
    private final ShopService service;

    @PostMapping("/customers")
    @ResponseStatus(HttpStatus.CREATED)
    public Customer createCustomer(
            @Valid @RequestBody CustomerRequest request) {
        return service.createCustomer(request);
    }

    @PostMapping("/credits")
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Object> createCredit(
            @Valid @RequestBody CreditRequest request) {
        return service.addCredit(request);
    }

    @PostMapping("/payments")
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Object> createPayment(
            @Valid @RequestBody PaymentRequest request) {
        return service.addPayment(request);
    }

    @GetMapping("/customers/{id}/ledger")
    public Map<String, Object> ledger(@PathVariable long id) {
        return service.ledger(id);
    }

    @GetMapping("/reports/dues")
    public List<Map<String, Object>> dues() {
        return service.dues();
    }
}
```

8.8 Security Configuration

Passwords are hashed with BCrypt, and every API except login and session check requires authentication.

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http,
        SecurityContextRepository contextRepository) throws Exception {
    http
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf
            .csrfTokenRepository(csrfRepository)
            .csrfTokenRequestHandler(csrfHandler))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET,
                "/api/auth/csrf", "/api/auth/session").permitAll()
            .requestMatchers(HttpMethod.POST,
                "/api/auth/login").permitAll()
            .anyRequest().authenticated())
        .exceptionHandling(e -> e.authenticationEntryPoint(
            new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
        .formLogin(form -> form.disable())
        .httpBasic(basic -> basic.disable());
    return http.build();
}
```

8.9 Reading Users from the Database (Spring Security)

Spring Security loads the user name, the BCrypt password hash and the role from the users table. The entered password is compared with the stored hash and is never stored as plain text.

```
@Bean
UserDetailsService userDetailsService(DataSource dataSource) {
    JdbcUserDetailsManager users = new JdbcUserDetailsManager(dataSource);
    users.setUsersByUsernameQuery(
        "SELECT username, password_hash, active "
        + "FROM users WHERE username = ?");
    users.setAuthoritiesByUsernameQuery(
        "SELECT username, CONCAT('ROLE_', role) "
        + "FROM users WHERE username = ?");
    return users;
}
```

8.10 Admin User Creation – BootstrapAdmin

When the application starts and the users table is empty, the first admin user is inserted with a BCrypt-hashed password taken from SHOP_BOOTSTRAP_PASSWORD (minimum 12 characters).

```
@Component
public class BootstrapAdmin implements CommandLineRunner {
    @Override
    public void run(String... args) {
        Integer users = jdbc.queryForObject(
            "SELECT COUNT(*) FROM users", Integer.class);
        if (users != null && users == 0) {
            if (password.length() < 12) {
                throw new IllegalStateException("Set "
                  + "SHOP_BOOTSTRAP_PASSWORD to at least 12 characters");
            }
            jdbc.update("INSERT INTO users "
                + "(username, password_hash, role) VALUES (?, ?, 'ADMIN')",
                username, passwordEncoder.encode(password));
        }
    }
}
```

8.11 Login API – AuthController

The login endpoint authenticates the user name and password against the database, creates a new session (changing the session ID) and returns 401 for wrong credentials.

```
@PostMapping("/login")
public Map<String, Object> login(@Valid @RequestBody LoginRequest request,
        HttpServletRequest httpRequest, HttpServletResponse httpResponse) {
    try {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.username(), request.password()));
        httpRequest.getSession(true);
        httpRequest.changeSessionId();
        SecurityContext context = SecurityContextHolder.createEmptyContext();
        context.setAuthentication(authentication);
        SecurityContextHolder.setContext(context);
        contextRepository.saveContext(context, httpRequest, httpResponse);
        return session(authentication);
    } catch (AuthenticationException exception) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
            "Username or password is incorrect");
    }
}
```

8.12 Database Scripts

SQL queries used to create the database and tables.

```
-- Database Creation
CREATE DATABASE smart_shop_db;
USE smart_shop_db;

-- Customers Table
CREATE TABLE customers (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    full_name   VARCHAR(120) NOT NULL,
    phone       VARCHAR(20)  NOT NULL,
    address     VARCHAR(255),
    active      BOOLEAN  NOT NULL DEFAULT TRUE,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated at  DATETIME NOT NULL DEFAULT CURRENT TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP
);

-- Credit Entries Table
CREATE TABLE credit_entries (
    id               BIGINT PRIMARY KEY AUTO_INCREMENT,
    customer_id      BIGINT NOT NULL,
    description      VARCHAR(255) NOT NULL,
    amount           DECIMAL(12,2) NOT NULL,
    transaction_date DATE NOT NULL,
    notes            VARCHAR(500),
    created_at       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

```
-- Payments Table
CREATE TABLE payments (
    id               BIGINT PRIMARY KEY AUTO_INCREMENT,
    customer_id      BIGINT NOT NULL,
    amount           DECIMAL(12,2) NOT NULL,
    transaction_date DATE NOT NULL,
    payment_method   VARCHAR(20) NOT NULL,
    notes            VARCHAR(500),
    request_key      VARCHAR(80) NOT NULL UNIQUE,
    created_at       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

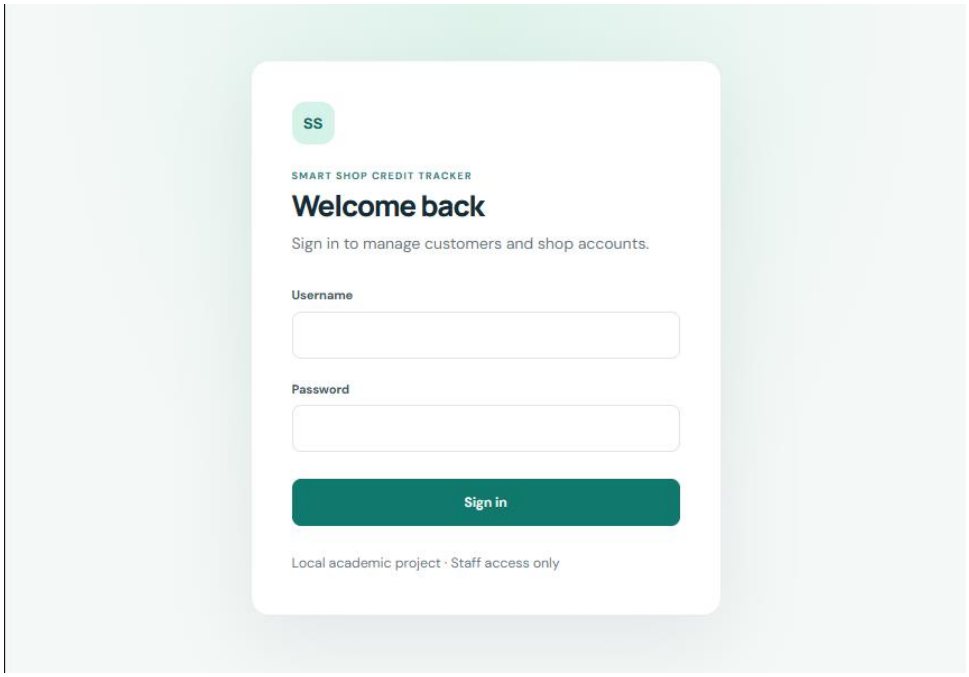
-- Users Table
CREATE TABLE users (
    username      VARCHAR(50) PRIMARY KEY,
    password_hash VARCHAR(100) NOT NULL,
    role          VARCHAR(20)  NOT NULL DEFAULT 'ADMIN',
    active        BOOLEAN NOT NULL DEFAULT TRUE
);
```

9. SAMPLE OUTPUT

The following screenshots show the actual execution of the Smart Shop Credit Tracker, captured from the web interface in the browser and from the MySQL command-line client during testing.

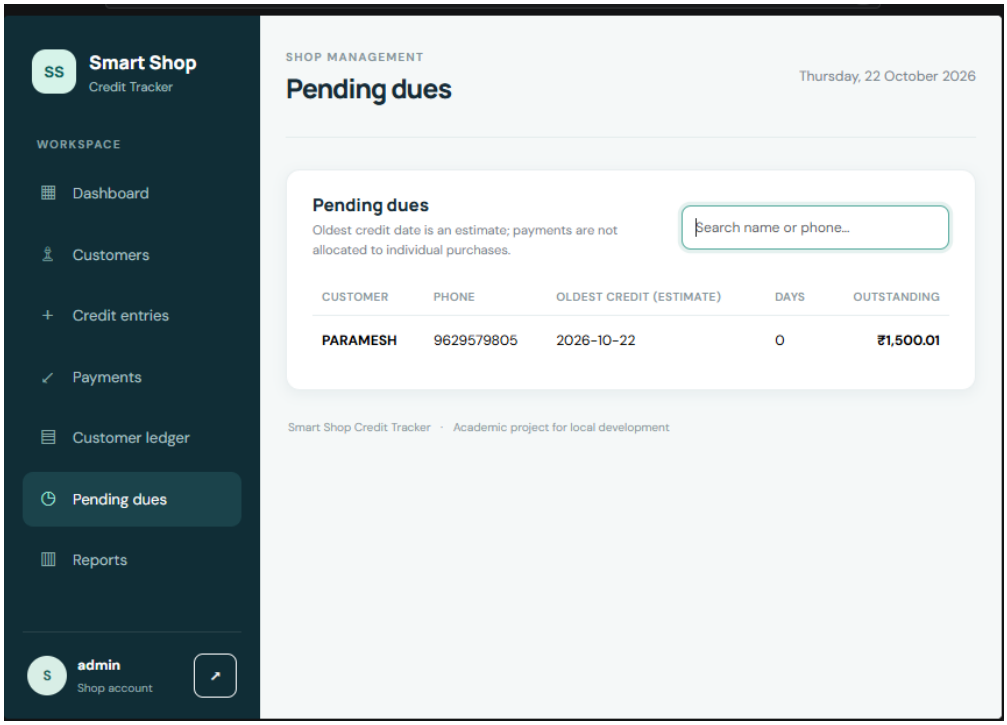
9.1 Login Page

The login page of the application. Only staff members with a valid username and password can access the system.



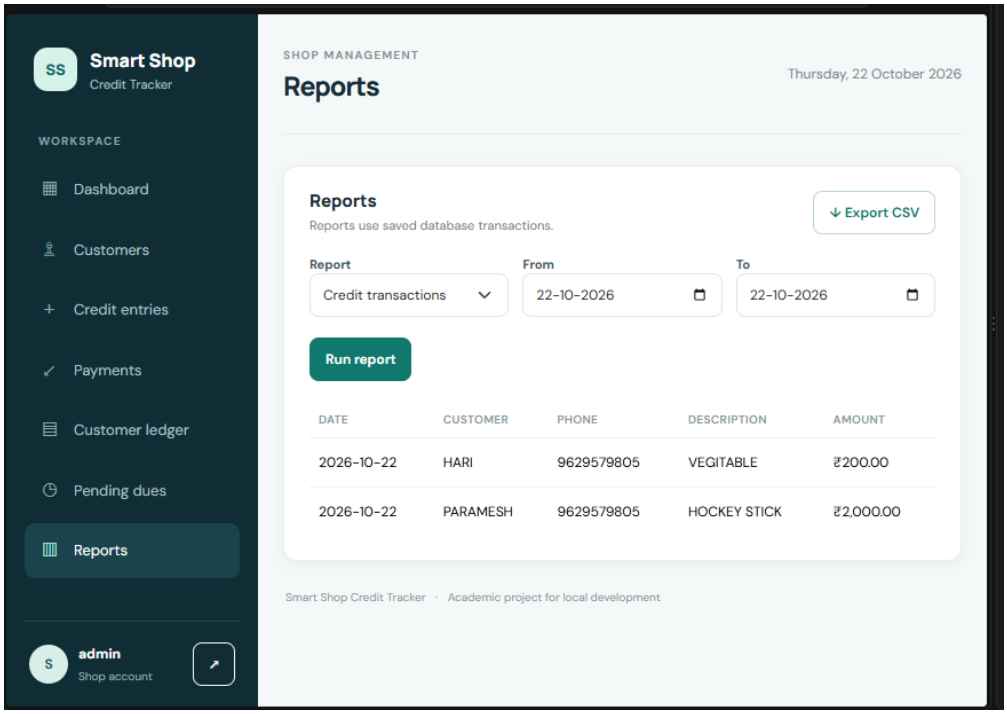
9.2 Pending Dues Page

Displays the customers who still owe money – customer PARAMESH (phone 9629579805) with the oldest credit date 2026-10-22 and an outstanding balance of ₹1,500.01.



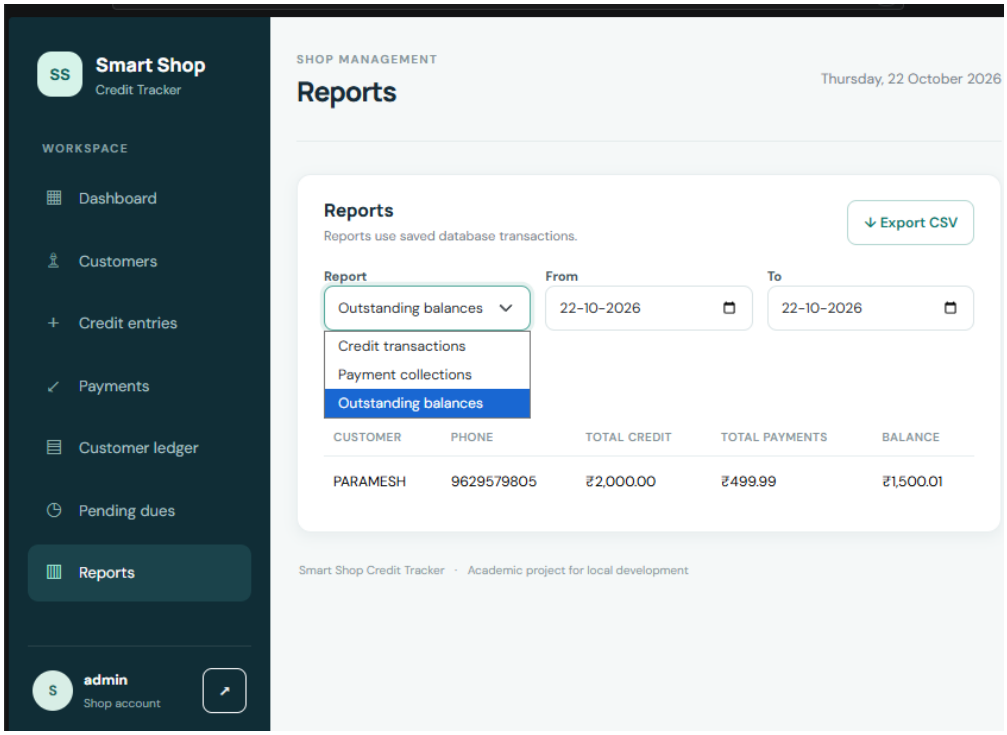
9.3 Credit Transactions Report

Shows the credit transactions of 22-10-2026 – VEGITABLE ₹200.00 for HARI and HOCKEY STICK ₹2,000.00 for PARAMESH – with the Export CSV option.



9.4 Outstanding Balances Report

The report type list (Credit transactions, Payment collections, Outstanding balances) and the outstanding balances report showing PARAMESH with a total credit of ₹2,000.00, total payments of ₹499.99 and a balance of ₹1,500.01.



9.5 Customers Table in MySQL Database

Shows the contents of the customers table verified using SELECT * FROM customers.

```
PS D:\Smart Shop Credit Tracker\backend> & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u smart_shop_app -p -D smart_s
hop_db
Enter password: ************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 45
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT * FROM customers;
+----+-----------+------------+------------+--------+---------------------+---------------------+
| id | full_name | phone      | address    | active | created_at          | updated_at          |
+----+-----------+------------+------------+--------+---------------------+---------------------+
|  1 | PARAMESH  | 9629579805 | ARANTHANGI |      1 | 2026-10-22 00:05:52 | 2026-10-22 00:05:52 |
|  2 | HARI      | 9629579805 | PUDUKKOTTAI |      1 | 2026-10-22 00:13:30 | 2026-10-22 00:13:30 |
+----+-----------+------------+------------+--------+---------------------+---------------------+
```

9.6 Credit Entries Table in MySQL Database

Shows the contents of the credit_entries table verified using SELECT * FROM credit_entries.

```
mysql> SELECT * FROM credit_entries;
+----+-------------+-------------+---------+------------------+-------+---------------------+
| id | customer_id | description | amount  | transaction_date | notes | created_at          |
+----+-------------+-------------+---------+------------------+-------+---------------------+
|  1 |           1 | HOCKEY STICK | 2000.00 | 2026-10-22       | NULL  | 2026-10-22 00:10:48 |
|  2 |           2 | VEGITABLE    |  200.00 | 2026-10-22       | NULL  | 2026-10-22 00:14:49 |
+----+-------------+-------------+---------+------------------+-------+---------------------+
2 rows in set (0.00 sec)
```

9.7 Payments Table in MySQL Database

Shows the contents of the payments table verified using SELECT * FROM payments. Both payments were made in CASH and each has a unique request key.

```
ysql> SELECT * FROM payments;
----+-------------+--------+------------------+----------------+-------+--------------------------------------+---------------------+
 id | customer_id | amount | transaction_date | payment_method | notes | request_key                          | created_at          |
----+-------------+--------+------------------+----------------+-------+--------------------------------------+---------------------+
  1 |           1 | 499.99 | 2026-10-22       | CASH           | NULL  | 7cd76614-509e-489b-9844-d9ffd10f9248 | 2026-10-22 00:11:05 |
  2 |           2 | 200.00 | 2026-10-22       | CASH           | NULL  | 5d3ebf06-069c-40ab-8dff-eb7966a07da5 | 2026-10-22 00:15:08 |
----+-------------+--------+------------------+----------------+-------+--------------------------------------+---------------------+
 rows in set (0.00 sec)
```

10. ANALYSIS

Testing of the Smart Shop Credit Tracker was carried out module by module to validate both functional correctness and overall system behavior.

10.1 Build and Execution

The back end was built and executed using the Maven Wrapper in the VS Code PowerShell terminal. The database password and the first admin password are supplied through environment variables (SHOP_DB_PASSWORD and SHOP_BOOTSTRAP_PASSWORD) before the application is started:

```
.\mvnw.cmd spring-boot:run
.\mvnw.cmd test
```

10.2 Test Cases and Results

TC No.	Test Case	Input	Expected Output	Actual Output	Result
TC01	Add Customer	Name: PARAMESH, Phone: 9629579805, Address: ARANTHANGI	Customer saved	Customer ID 1 stored in customers table	PASS
TC02	Add Customer	Name: HARI, Phone: 9629579805, Address: PUDUKKOTTAI	Customer saved	Customer ID 2 stored in customers table	PASS
TC03	Record Credit	Customer: PARAMESH, Item: HOCKEY STICK, Amount: 2000.00	Credit saved	Entry ID 1 with ₹2000.00 stored in credit_entries	PASS
TC04	Record Credit	Customer: HARI, Item: VEGITABLE, Amount: 200.00	Credit saved	Entry ID 2 with ₹200.00 stored in credit_entries	PASS
TC05	Record Payment	Customer: PARAMESH, Amount: 499.99, Method: CASH	Payment saved, balance ₹1500.01	Payment ID 1 stored; balance shown as ₹1,500.01	PASS
TC06	Record Payment	Customer: HARI, Amount: 200.00, Method: CASH	Payment saved, balance ₹0.00	Payment ID 2 stored; HARI not listed in dues	PASS
TC07	Pending Dues	Open Pending dues page	Only customers with balance > 0	PARAMESH – ₹1,500.01 listed	PASS
TC08	Credit Report	From: 22-10-2026, To: 22-10-2026	All credits of the day	HARI ₹200.00 and PARAMESH ₹2,000.00 listed	PASS
TC09	Outstanding Report	Report: Outstanding balances	Customers with outstanding balance	PARAMESH: 2000.00 – 499.99 = ₹1,500.01	PASS
TC10	Payment above balance (JUnit)	Balance: 1200.00, Payment: 1200.01	BusinessException; no payment inserted	Checked in ShopServiceTest	JUnit test
TC11	Partial payment (JUnit)	Balance: 2000.00, Payment: 800.00	Payment saved, balance 1200.00	Checked in ShopServiceTest	JUnit test
TC12	Full payment (JUnit)	Balance: 1200.00, Payment: 1200.00	Balance becomes 0	Checked in ShopServiceTest	JUnit test

TC No.	Test Case	Input	Expected Output	Actual Output	Result
TC13	Duplicate submission (JUnit)	Same request key submitted again	"This payment was already recorded"; no new insert	Checked in ShopServiceTest	JUnit test
TC14	Amount validation (JUnit)	Amount: 0, -1.00 and 1.001	Validation errors	Checked in ShopServiceTest	JUnit test

10.3 Database Verification

The data stored in the MySQL database was verified using the following SQL queries:

```
USE smart_shop_db;
SELECT * FROM customers;
SELECT * FROM credit_entries;
SELECT * FROM payments;
```

The queries confirmed that all customer details, credit entries and payments were properly stored and maintained in the MySQL database, demonstrating successful JDBC connectivity and data persistence (Sections 9.5 to 9.7).

10.4 Analysis

Functional analysis: All the tested operations – adding customers, recording credit, recording payments, pending dues and the credit and outstanding reports – behaved as expected. The JUnit test cases cover the payment rules, including the edge cases of a payment larger than the balance, a full payment, a duplicate submission and invalid amounts.

Data integrity analysis: The balances shown in the application matched the values stored in the tables. For PARAMESH, the total credit of ₹2,000.00 minus the payment of ₹499.99 gives the balance of ₹1,500.01 shown in the pending dues and outstanding reports, and HARI's credit of ₹200.00 is cleared by the payment of ₹200.00. Because the balance is calculated from the transactions and not stored separately, it cannot go out of sync.

Security analysis: Passwords are stored as BCrypt hashes, all APIs except login require an authenticated session, CSRF tokens are checked for modifying requests, SQL queries use parameter placeholders, and the database password is read from environment variables instead of the source code.

Usability analysis: The web interface has a simple sidebar with the Dashboard, Customers, Credit entries, Payments, Customer ledger, Pending dues and Reports pages. Each operation needs only a few inputs and gives an immediate response message, and the search box and CSV export make daily work easier.

Comparative analysis: Compared with the existing notebook method, the application removes paper-based record keeping, provides instant balances and pending dues, maintains a dated record of every credit and payment, validates every entry and protects the data with a login, thereby reducing human errors and saving time.

Overall, the analysis confirms that the system meets its objectives of reducing manual effort, improving the accuracy of credit records and providing a reliable platform for managing customer credit in a shop.

11. CONCLUSION

The Summer Internship at **Astonish InfoTech Private Limited, Tiruchirappalli** was successfully completed from 01/06/2026 to 30/06/2026 in the domain of Java Development. During this one-month internship, comprehensive knowledge of Core Java programming was gained through structured learning and hands-on practice, including OOP principles, Exception Handling, the Collections Framework, JDBC database connectivity and the Spring Boot framework.

The **Smart Shop Credit Tracker** was successfully developed using Java, Spring Boot and a MySQL database and implements all the required credit management operations. The system demonstrates CRUD operations on customer records, credit and payment recording, automatic balance calculation, a customer ledger with running balance, pending dues and date-wise reports, validation of payments against the outstanding balance, protection against duplicate payments, and secure staff login with proper exception handling.

The internship also provided valuable practical experience in REST API development, database design, transaction management, application security and software testing. It improved my problem-solving and analytical thinking abilities and helped me understand industry-oriented development practices. The developed system can serve as a foundation for a more advanced shop management solution in the future.

12. FUTURE WORK

- Add **SMS or WhatsApp reminders** to customers who have pending dues.
- Allocate payments to **individual credit entries** so that the exact age of every due amount is known.
- Add **due dates and credit limits** for each customer with warnings when the limit is crossed.
- Support **multiple staff users with different roles** and a record of who made each entry.
- Generate **customer statements and reports in PDF** format.
- Add **charts and graphs** to the dashboard for monthly credit and collection trends.
- Add **stock, billing and invoice** modules to make a complete shop management system.
- Develop a **mobile application** for the shop owner using the same REST APIs.
- Use **database migration tools** such as Flyway and integration tests with a test database.
- Provide **automatic database backup** and deploy the application on a cloud platform such as AWS or Azure.

13. REFERENCES

1. Herbert Schildt, “Java: The Complete Reference”, 11th Edition, McGraw-Hill Education, 2019.
2. Cay S. Horstmann, “Core Java Volume I – Fundamentals”, 11th Edition, Prentice Hall, 2018.
3. Bruce Eckel, “Thinking in Java”, 4th Edition, Prentice Hall, 2006.
4. Oracle Corporation, “Java SE 17 Documentation”, Available at: https://docs.oracle.com/javase/17/ [Accessed: June 2026].
5. Oracle Corporation, “MySQL 8.0 Reference Manual”, Available at: https://dev.mysql.com/doc/refman/8.0/en/ [Accessed: June 2026].
6. Oracle Corporation, “JDBC Database Access Tutorial”, Available at: https://docs.oracle.com/javase/tutorial/jdbc/ [Accessed: June 2026].
7. Spring, “Spring Boot Reference Documentation”, Available at: https://docs.spring.io/spring-boot/ [Accessed: June 2026].

8. Spring, “Spring Framework – Data Access with JDBC”, Available at: https://docs.spring.io/spring-framework/reference/data-access/jdbc.html [Accessed: June 2026].
9. Spring, “Spring Security Reference”, Available at: https://docs.spring.io/spring-security/reference/ [Accessed: June 2026].
10. Astonish InfoTech Private Limited, Available at: https://www.astonishinfotech.com [Accessed: June 2026].